

Documentação: Lógica de Etapa Atual e Status

Índice

1. Visão Geral
 2. Campos de Entrada Esperados
 3. Função: calcular_etapa_atual()
 4. Função: calcular_status()
 5. Tratamento de Tratamentos Térmicos
 6. Exemplos Práticos
 7. Fluxograma de Decisão
-

Visão Geral

O sistema calcula automaticamente:

- **Etapa Atual:** qual a próxima atividade que precisa ser executada
- **Status:** se essa atividade está no prazo, em potencial atraso, ou atrasada

Conceito Fundamental

- **Etapa Atual** $\neq$ Última etapa concluída
- **Etapa Atual** = Próxima etapa a ser executada

Exemplo: Se Fusão foi concluída, a **Etapa Atual** é "Desmoldagem" (a próxima), não "Fusão" (a última feita).

Campos de Entrada Esperados

Dados obrigatórios de cada linha (row):

```
python
```

```
# Datas previstas (planejamento)
'projeto_prev', 'fabr_modelo_prev', 'insp_modelo_prev',
'ficha_processo_prev', 'modelacao_prev', 'lib_processo_eng_prev',
'moldagem_prev', 'fusao_prev', 'desmoldagem_prev',
'normalizacao_corte_prev', 'normalizacao_prev', 'tempera_prev',
'revenimento_prev', 'alivio_tensoes_prev',
'acabamento_prev', 'usinagem_prev', 'lib_fundido_prev'

# Datas reais (execução)
'projeto_real', 'fabr_modelo_real', 'insp_modelo_real',
'ficha_processo_real', 'modelacao_real', 'lib_processo_eng_real',
'moldagem_real', 'fusao_real', 'desmoldagem_real',
'normalizacao_corte_real', 'normalizacao_real', 'tempera_real',
'revenimento_real', 'alivio_tensoes_real',
'acabamento_real', 'usinagem_real', 'lib_fundido_real'

# Prazos de referência
'prz_fundicao'      # Prazo de fundição
'prz_contratual'   # Prazo contratual
```

Formato esperado:

- **Datas:** `pd.Timestamp` ou `datetime`, ou `None`/`NaN` se não preenchido
- **Prazos:** `pd.Timestamp` ou `datetime`

 **Função:** `calcular_etapa_atual(row)`

O que faz:

Determina qual a próxima etapa que precisa ser executada, verificando as etapas **de trás para frente** (do fim para o início do processo).

Lógica passo a passo:

python

```
def calcular_etapa_atual(row):
    """
    Retorna a próxima etapa que precisa ser executada.
    Verifica as etapas de trás para frente (do fim para o início).
    """

    # 1. Se Liberação Fundido foi feita → FINALIZADO
    if pd.isna(row.get('lib_fundido_real')):
        return 'Finalizado'

    # 2. Se Usinagem foi feita → próxima é Liberação Fundido
    if pd.isna(row.get('usinagem_real')):
        return 'Liberação Fundido'

    # 3. Se Acabamento foi feito → próxima é Usinagem
    if pd.isna(row.get('acabamento_real')):
        return 'Usinagem'

    # 4. LÓGICA COMPLEXA: Acabamento ou Tratamento Térmico?
    # Se Desmoldagem foi feita...
    if pd.isna(row.get('desmoldagem_real')):
        # Verifica se TODOS os TTs planejados foram executados
        if todos_tt_concluidos_ou_nao_aplicaveis(row):
            return 'Acabamento'

    # 5. Se algum TT foi feito OU está planejado → próxima é TT
    if algum_tt_real_preenchido(row):
        return 'Tratamento Térmico'

    if desmoldagem_feita_e_tt_planejado(row):
        return 'Tratamento Térmico'

    # 6. Se Desmoldagem feita mas NENHUM TT planejado → pula para Acabamento
    if desmoldagem_feita_e_nenhum_tt_planejado(row):
        return 'Acabamento'

    # 7. Se Fusão foi feita → próxima é Desmoldagem
    if pd.isna(row.get('fusao_real')):
        return 'Desmoldagem'

    # 8-15. Continua verificando de trás para frente...
    if pd.isna(row.get('moldagem_real')):
        return 'Fusão'
```

```
if pd.isna(row.get('lib_processo_eng_real')):  
    return 'Moldagem'  
  
# ... e assim por diante até Projeto Eng  
  
# 16. Se nada foi feito mas tem Projeto Eng planejado  
if pd.isna(row.get('projeto_prev')):  
    return 'Projeto Eng'  
  
# 17. Caso contrário  
return 'Planejamento Não Preenchido'
```

Funções auxiliares para Tratamentos Térmicos:

python

```

def todos_tt_concluidos_ou_nao_aplicaveis(row):
    """
    Verifica se TODOS os TTs que foram planejados (Prev.) já foram executados (Real)
    OU se não eram aplicáveis (Prev. vazio significa que não precisa fazer).
    """
    tts = [
        ('alivio_tensoes_prev', 'alivio_tensoes_real'),
        ('revenimento_prev', 'revenimento_real'),
        ('tempera_prev', 'tempera_real'),
        ('normalizacao_prev', 'normalizacao_real'),
        ('normalizacao_corte_prev', 'normalizacao_corte_real')
    ]

    for prev, real in tts:
        # Se FOI planejado (Prev. preenchido)...
        if pd.notna(row.get(prev)):
            # ... mas NÃO foi executado (Real vazio) → FALSO
            if pd.isna(row.get(real)):
                return False

    return True # Todos os TTs planejados foram feitos OU não havia TTs planejados

def algum_tt_real_preenchido(row):
    """Verifica se algum TT já foi executado (Real preenchido)."""
    tts_real = ['normalizacao_corte_real', 'normalizacao_real',
                'tempera_real', 'revenimento_real', 'alivio_tensoes_real']
    return any(pd.notna(row.get(tt)) for tt in tts_real)

def desmoldagem_feita_e_tt_planejado(row):
    """Desmoldagem feita E existe algum TT planejado (Prev. preenchido)."""
    if pd.isna(row.get('desmoldagem_real')):
        return False

    tts_prev = ['normalizacao_corte_prev', 'normalizacao_prev',
                'tempera_prev', 'revenimento_prev', 'alivio_tensoes_prev']
    return any(pd.notna(row.get(tt)) for tt in tts_prev)

def desmoldagem_feita_e_nenhum_tt_planejado(row):
    """Desmoldagem feita E NENHUM TT foi planejado (todos Prev. vazios)."""
    if pd.isna(row.get('desmoldagem_real')):
        return False

    tts_prev = ['normalizacao_corte_prev', 'normalizacao_prev',

```

```
        'tempera_prev', 'revenimento_prev', 'alivio_tensoes_prev']  
    return all(pd.isna(row.get(tt)) for tt in tts_prev)
```

 **Função:** `calcular_status(row, etapa_atual, hoje)`

O que faz:

Determina se a etapa atual está no prazo, em potencial atraso, ou atrasada.

Lógica passo a passo:

python

```
def calcular_status(row, etapa_atual, hoje):
    """
    Calcula o status baseado na data prevista da etapa atual
    e nos prazos de referência.

    Args:
        row: linha do DataFrame com todas as datas
        etapa_atual: string retornada por calcular_etapa_atual()
        hoje: pd.Timestamp de hoje

    Returns:
        'Finalizado' | 'No Prazo' | 'Potencial Atraso' | 'Atrasado'
    """

    # 1. Se finalizado → status = Finalizado
    if etapa_atual == 'Finalizado':
        return 'Finalizado'

    # 2. Busca a data prevista da etapa atual
    data_prevista = obter_data_prevista_da_etapa(row, etapa_atual)

    # 3. Define o prazo de referência
    prazo_referencia = obter_prazo_referencia(row)

    # 4. Se não tem data prevista OU data prevista = infinito → No Prazo
    if data_prevista is None or data_prevista == INFINITO:
        return 'No Prazo'

    # 5. Se data prevista ainda não passou → No Prazo
    if data_prevista >= hoje:
        return 'No Prazo'

    # 6. Data prevista JÁ PASSOU (< hoje)
    # Verifica o prazo de referência:
    if prazo_referencia < hoje:
        return 'Atrasado' # Prazo de referência também passou
    else:
        return 'Potencial Atraso' # Prazo de referência ainda não passou
```

Funções auxiliares:

python

```

def obter_data_prevista_da_etapa(row, etapa):
    """
    Retorna a data prevista correspondente à etapa atual.
    Para Tratamento Térmico, retorna a MENOR data entre os TTs pendentes.
    """

    mapeamento = {
        'Projeto Eng': 'projeto_prev',
        'Fabricação Modelo': 'fabr_modelo_prev',
        'Inspeção Modelo': 'insp_modelo_prev',
        'Ficha Processo': 'ficha_processo_prev',
        'Modelação Processo': 'modelacao_prev',
        'Liberação Processo Eng': 'lib_processo_eng_prev',
        'Moldagem': 'moldagem_prev',
        'Fusão': 'fusao_prev',
        'Desmoldagem': 'desmoldagem_prev',
        'Acabamento': 'acabamento_prev',
        'Usinagem': 'usinagem_prev',
        'Liberação Fundido': 'lib_fundido_prev',
    }

    # Caso especial: Tratamento Térmico
    if etapa == 'Tratamento Térmico':
        return obter_menor_data_tt_pendente(row)

    # Etapas normais
    campo = mapeamento.get(etapa)
    if campo:
        return row.get(campo)

    return None

def obter_menor_data_tt_pendente(row):
    """
    Retorna a MENOR data entre os TTs que:
    - Têm data prevista (Prev. preenchido) E
    - NÃO têm data real (Real vazio)

    Retorna INFINITO se não há TTs pendentes.
    """
    INFINITO = pd.Timestamp('2099-12-31')

    tts = [

```

```

        ('normalizacao_corte_prev', 'normalizacao_corte_real'),
        ('normalizacao_prev', 'normalizacao_real'),
        ('tempera_prev', 'tempera_real'),
        ('revenimento_prev', 'revenimento_real'),
        ('alivio_tensoes_prev', 'alivio_tensoes_real')
    ]

    datas_pendentes = []

    for prev, real in tts:
        # Se tem Prev. mas não tem Real → está pendente
        if pd.notna(row.get(prev)) and pd.isna(row.get(real)):
            datas_pendentes.append(row.get(prev))

    if not datas_pendentes:
        return INFINITO # Nenhum TT pendente

    return min(datas_pendentes)

def obter_prazo_referencia(row):
    """
    Define qual prazo usar como referência:
    - Se Usinagem|Previsto preenchido → Prazo Contratual
    - Senão → Prazo Fundição
    """
    if pd.notna(row.get('usinagem_prev')):
        return row.get('prz_contratual')
    else:
        return row.get('prz_fundicao')

```

🔥 Tratamento de Tratamentos Térmicos

Por que é complexo?

Uma peça pode ter **0 a 5 tipos de TT** planejados:

1. Normalização Corte
2. Normalização
3. Têmpera/Solubilização
4. Revenimento
5. Alívio de Tensões

Nem todas as peças precisam de todos os TTs.

Regras:

1. TT é necessário?

- Se **Prev.** está **preenchido** → SIM, é necessário
- Se **Prev.** está **vazio** → NÃO, não é aplicável

2. TT foi concluído?

- Se **Real** está **preenchido** → SIM, foi feito
- Se **Real** está **vazio** → NÃO, ainda pendente

3. Etapa Atual = "Tratamento Térmico" quando:

- Desmoldagem foi feita E
- Existe pelo menos 1 TT planejado (Prev. preenchido) que não foi feito (Real vazio)

4. Etapa Atual pula TT e vai para "Acabamento" quando:

- Desmoldagem foi feita E
- TODOS os TTs planejados foram concluídos (Real preenchido) OU
- NENHUM TT foi planejado (todos Prev. vazios)

5. Status quando Etapa = "Tratamento Térmico":

- Data prevista = **MENOR data** entre os TTs pendentes
- Exemplo: se Normalização|Prev = 15/05 e Têmpera|Prev = 20/05, mas só Normalização está pendente → data prevista = 15/05

💡 Exemplos Práticos

Exemplo 1: Peça em Fusão (simples)

Entrada:

```
python

moldagem_real = '2026-05-10'
fusao_prev = '2026-05-15'
fusao_real = None # Não foi feito ainda
prz_fundicao = '2026-05-25'
hoje = '2026-05-18'
```

Saída:

```
python
```

```
etapa_atual = 'Fusão'           # Moldagem foi feita, próxima é Fusão
data_prevista = '2026-05-15'    # Fusão|Prev
prazo_ref = '2026-05-25'        # Prazo Fundição
status = 'Potencial Atraso'    # 15/05 < 18/05 (passou), mas 25/05 > 18/05 (prazo 0
```

Exemplo 2: Peça com TT planejado

Entrada:

```
python

fusao_real = '2026-05-10'
desmoldagem_real = '2026-05-11'
normalizacao_prev = '2026-05-15'
normalizacao_real = None
tempera_prev = '2026-05-20'
tempera_real = None
prz_fundicao = '2026-06-01'
hoje = '2026-05-18'
```

Saída:

```
python

etapa_atual = 'Tratamento Térmico' # Desmoldagem feita + TT planejado
data_prevista = '2026-05-15'        # MENOR entre Norm(15/05) e Têmpera(20/05)
prazo_ref = '2026-06-01'            # Prazo Fundição
status = 'Potencial Atraso'        # 15/05 < 18/05, mas 01/06 > 18/05
```

Exemplo 3: Peça SEM TT planejado

Entrada:

```
python
```

```
fusao_real = '2026-05-10'
desmoldagem_real = '2026-05-11'
normalizacao_prev = None # Sem TT
tempera_prev = None
revenimento_prev = None
alivio_tensoes_prev = None
normalizacao_corte_prev = None
acabamento_prev = '2026-05-20'
acabamento_real = None
hoje = '2026-05-18'
```

Saída:

```
python

etapa_atual = 'Acabamento' # Desmoldagem feita + NENHUM TT planejado → pula TT
data_prevista = '2026-05-20' # Acabamento|Prev
status = 'No Prazo' # 20/05 > 18/05
```

Exemplo 4: Peça com TT parcialmente concluído

Entrada:

```
python

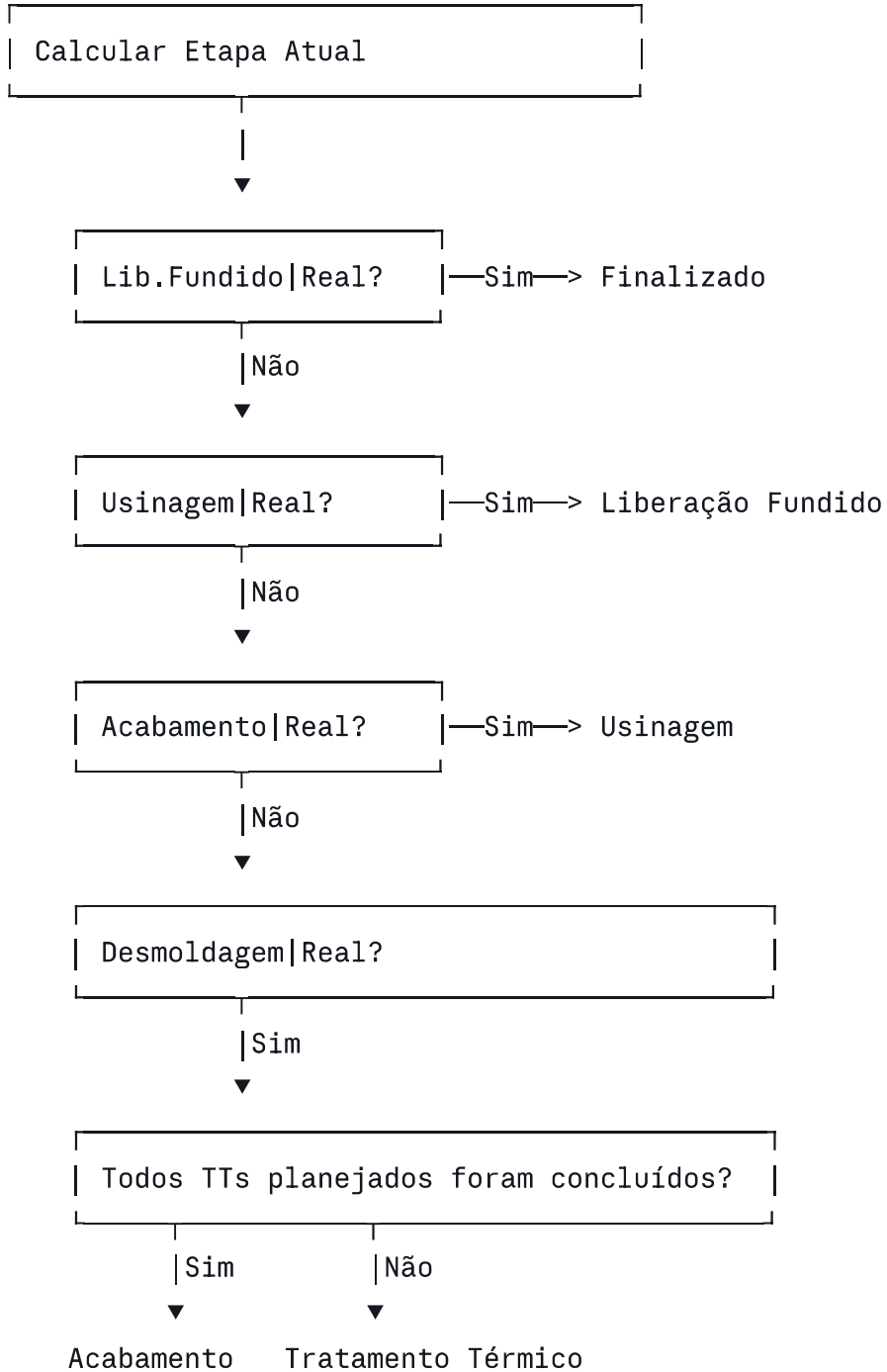
fusao_real = '2026-05-10'
desmoldagem_real = '2026-05-11'
normalizacao_prev = '2026-05-15'
normalizacao_real = '2026-05-16' # JÁ FOI FEITO
tempera_prev = '2026-05-20'
tempera_real = None # PENDENTE
prz_fundicao = '2026-06-01'
hoje = '2026-05-18'
```

Saída:

```
python

etapa_atual = 'Tratamento Térmico' # Ainda tem TT pendente (Têmpera)
data_prevista = '2026-05-20' # Têmpera|Prev (Normalização já feita, não conta)
status = 'No Prazo' # 20/05 > 18/05
```

Fluxograma de Decisão



(continua para Fusão, Moldagem, etc...)

Integração no Código

Onde entra no projeto:

Arquivo modificado: `atualizar_v12_completo.py`

Localização: Função `processar()`, ao gerar `acompany_records`

Antes:

```
python

# Cálculo simplificado
status_acomp = 'Fundido' if fusao_real else 'Planejado'
etapa = 'Fusão'
```

Depois:

```
python

# Cálculo com lógica completa do Excel
etapa_atual = calcular_etapa_atual(r)
status_acomp = calcular_status(r, etapa_atual, hoje)
```

Checklist de Validação

Para verificar se a implementação está correta:

- ☐ Peça finalizada (Lib.Fundido|Real preenchido) → Etapa = "Finalizado"
 - ☐ Peça com TT planejado mas não executado → Etapa = "Tratamento Térmico"
 - ☐ Peça sem TT planejado → Etapa pula de Desmoldagem para Acabamento
 - ☐ Status "Potencial Atraso" quando data prevista passou mas prazo de referência não
 - ☐ Status "Atrasado" quando ambos (data prevista E prazo referência) passaram
 - ☐ TT com múltiplas datas → usa a MENOR data entre os pendentes
 - ☐ Prazo de referência = Contratual se Usinagem|Prev preenchido, senão Fundição
-

Última atualização: 19/05/2026

Versão: 1.0

Autor: Sistema de Gestão Fundição Moreno